

Classe : LAII-1A, LAII-1B, LAII-1C, LAII-1D & LAII-1E	Devoir Surveillé	Nombre de pages : 2
	Date : 17 Mars 2018	Durée : 1h30
Barème approximatif : (5,7.5,7.5)	Documents : non autorisé	Calculatrice : autorisée

Structure des ordinateurs

Question de cours (5 points)

- Dans un montage à base d'un microcontrôleur PIC18f4550, L'horloge externe utilisée est de fréquence 8MHz.
 - Quel est la valeur de la fréquence de l'horloge instruction de ce PIC ? (1pt)
 - Calculer alors la durée d'un cycle machine. (1pt)
- Quelles sont les caractéristiques de la RAM du pic18f4550 (sa taille, nombre des fils d'adresse et nombre des bits des données dans une case mémoire) (0.75pt)
- Donnez le principe l'architecture de HARVARD et les principaux éléments qui la compose? (1.5pt)
- Définir les mots : SFR, BSR. (0.5pt)

Exercice1 : (7.5points)

Soient les deux équations suivantes :

$$E1 = (A1 - B1) + C1 \quad \text{et} \quad E2 = (\overline{A1} - B1) \oplus \overline{C1}$$

E1, E2, A1, B1 et C1 sont des variables dont les adresses sont comme suit :

$$E1=0x42 \quad E2=0x512 \quad A1= 0x30 \quad B1=0x31 \quad C1=0x2FA$$

- Mettre les adresses des variables A1, B1, C1, E1 et E2 sur le schéma de la mémoire (Annexe1) en indiquant le numéro de la banque pour chaque variable. (1.25pt)
- Dans un premier temps, on désire placer les valeurs 0x05, 0x02 et 0x10 respectivement dans les variables A1, B1 et C1.
Ecrire le programme en assembleur qui permet de déclarer les 5 variables et de placer les valeurs correspondantes dans A1, B1 et C1. (1.75pt)
- Ecrire un programme qui réalise l'équation E1. Calculer le temps nécessaire pour l'exécution de ce programme sachant que l'horloge externe est de fréquence $F_{osc} = 10\text{MHz}$. (2pt)
- Ecrire un programme qui réalise l'équation E2 (On pourra utiliser l'instruction complément pour écrire $\overline{A1}$ et $\overline{C1}$). (2pt)
- Que doit être le contenu de E1 et E2 après l'exécution du programme. (0.5pt)

Exercice 2 : (7,5points)

En se référant aux documents de l'**annexe2**, Écrire un programme en assembleur qui permet, à partir d'une valeur « 1 ou 2 » enregistrée en un registre appelé « Var1 », soit de calculer la somme des contenues des 3 cases successives de la mémoire RAM, soit d'ajouter à une case mémoire la valeur « 5 » tant que le résultat est inférieur à 255 ou il n'a pas un passage de retenue, en suivant les questions suivants :

- 1) Commencer le programme à partir de l'adresse 0x0C de la mémoire programme. **(0.5pt)**
- 2) Déclarer 3 variables Var1, Var2, Var3 et Var4 dont leurs adresses sont successives et commencent à partir de « 0x4d ». **(0.5pt)**
- 3) Sous l'étiquette « comparaison », Comparer le contenu du registre « Var1 » avec les valeurs 1 ou 2 : **(2.5pt)**
 - S'il est égal à «1» passer à l'étiquette « somme »
 - S'il est égal à «2» passer à l'étiquette « ajout »
 - Si non revenir au label « comparaison ».
- 4) Sous l'étiquette « somme », Calculer la somme des 3 variables (Var2, Var3 et Var4) et placer le résultat dans « Var2 ». **(1.5pt)**
- 5) Sous l'étiquette « ajout », ajouter à « Var 3 » la valeur « 5 » tant que le résultat est inférieur à 255 ou il n'y a pas un passage de retenue. **(2.5pt)**

Annexe 2

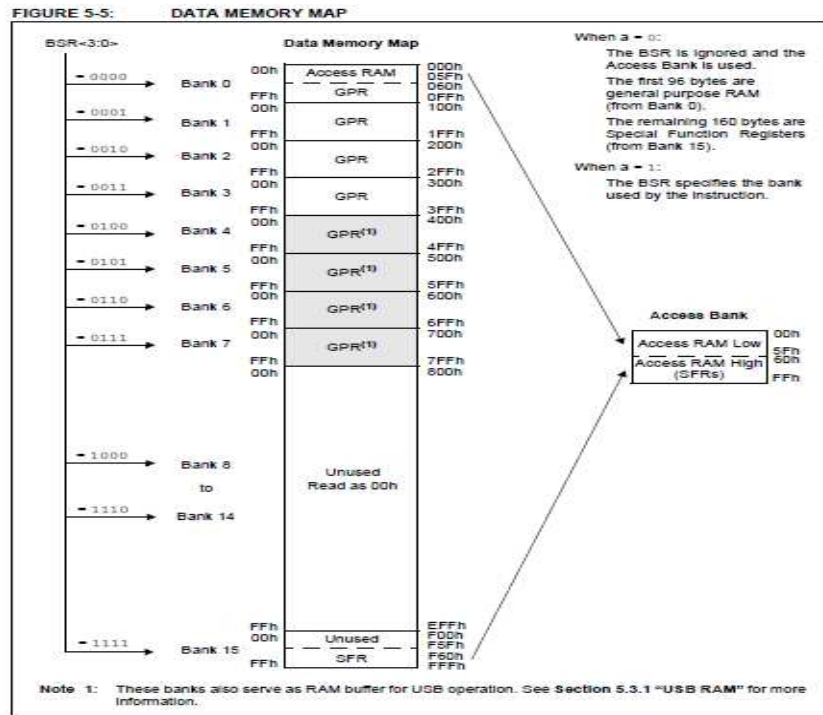


TABLE 26-2: PIC18FXXXX INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word		Status Affected	Notes
			MSb	LSb		
BYTE-ORIENTED OPERATIONS						
ADDWF f, d, a	Add WREG and f	1	0010	01da ffff ffff	C, DC, Z, OV, N	1, 2
ADDWFC f, d, a	Add WREG and Carry bit to f	1	0010	00da ffff ffff	C, DC, Z, OV, N	1, 2
ANDWF f, d, a	AND WREG with f	1	0001	01da ffff ffff	Z, N	1, 2
CLRF f, a	Clear f	1	0110	101a ffff ffff	Z	2
COMF f, d, a	Complement f	1	0001	11da ffff ffff	Z, N	1, 2
CPFSEQ f, a	Compare f with WREG, Skip =	1 (2 or 3)	0110	001a ffff ffff	None	4
CPFSGT f, a	Compare f with WREG, Skip >	1 (2 or 3)	0110	010a ffff ffff	None	4
CPFSLT f, a	Compare f with WREG, Skip <	1 (2 or 3)	0110	000a ffff ffff	None	1, 2
DECf f, d, a	Decrement f	1	0000	01da ffff ffff	C, DC, Z, OV, N	1, 2, 3, 4
DECFSZ f, d, a	Decrement f, Skip if 0	1 (2 or 3)	0010	11da ffff ffff	None	1, 2, 3, 4
DCFSNZ f, d, a	Decrement f, Skip if Not 0	1 (2 or 3)	0100	11da ffff ffff	None	1, 2
INCF f, d, a	Increment f	1	0010	10da ffff ffff	C, DC, Z, OV, N	1, 2, 3, 4
INCFSZ f, d, a	Increment f, Skip if 0	1 (2 or 3)	0011	11da ffff ffff	None	4
INFSNZ f, d, a	Increment f, Skip if Not 0	1 (2 or 3)	0100	10da ffff ffff	None	1, 2
IORWF f, d, a	Inclusive OR WREG with f	1	0001	00da ffff ffff	Z, N	1, 2
MOVF f, d, a	Move f	1	0101	00da ffff ffff	Z, N	1
MOVFF f _s , f _d	Move f _s (source) to 1st word f _d (destination) 2nd word	2	1100	ffff ffff ffff	None	
MOVWF f, a	Move WREG to f	1	0110	111a ffff ffff	None	
MULWF f, a	Multiply WREG with f	1	0000	001a ffff ffff	None	1, 2
NEGF f, a	Negate f	1	0110	110a ffff ffff	C, DC, Z, OV, N	
RLCF f, d, a	Rotate Left f through Carry	1	0011	01da ffff ffff	C, Z, N	1, 2
RLNCF f, d, a	Rotate Left f (No Carry)	1	0100	01da ffff ffff	Z, N	
RRCF f, d, a	Rotate Right f through Carry	1	0011	00da ffff ffff	C, Z, N	
RRNCF f, d, a	Rotate Right f (No Carry)	1	0100	00da ffff ffff	Z, N	
SETF f, a	Set f	1	0110	100a ffff ffff	None	1, 2
SUBFWB f, d, a	Subtract f from WREG with Borrow	1	0101	01da ffff ffff	C, DC, Z, OV, N	
SUBWF f, d, a	Subtract WREG from f	1	0101	11da ffff ffff	C, DC, Z, OV, N	1, 2
SUBWFB f, d, a	Subtract WREG from f with Borrow	1	0101	10da ffff ffff	C, DC, Z, OV, N	
SWAPF f, d, a	Swap Nibbles in f	1	0011	10da ffff ffff	None	4
TSTFSZ f, a	Test f, Skip if 0	1 (2 or 3)	0110	011a ffff ffff	None	1, 2
XORWF f, d, a	Exclusive OR WREG with f	1	0001	10da ffff ffff	Z, N	

BIT-ORIENTED OPERATIONS									
BCF	f, b, a	Bit Clear f	1	1001	bbba	ffff	ffff	None	1, 2
BSF	f, b, a	Bit Set f	1	1000	bbba	ffff	ffff	None	1, 2
BTFSC	f, b, a	Bit Test f, Skip if Clear	1 (2 or 3)	1011	bbba	ffff	ffff	None	3, 4
BTFSS	f, b, a	Bit Test f, Skip if Set	1 (2 or 3)	1010	bbba	ffff	ffff	None	3, 4
BTG	f, d, a	Bit Toggle f	1	0111	bbba	ffff	ffff	None	1, 2

Mnemonic, Operands		Description	Cycles	16-Bit Instruction Word				Status Affected	Notes
				MSb		LSb			
LITERAL OPERATIONS									
ADDLW	k	Add Literal and WREG	1	0000	1111	kkkk	kkkk	C, DC, Z, OV, N	
ANDLW	k	AND Literal with WREG	1	0000	1011	kkkk	kkkk	Z, N	
IORLW	k	Inclusive OR Literal with WREG	1	0000	1001	kkkk	kkkk	Z, N	
LFSR	f, k	Move Literal (12-bit) 2nd word to FSR(f) 1st word	2	1110	1110	00ff	kkkk	None	
MOVLB	k	Move Literal to BSR<3:0>	1	0000	0001	0000	kkkk	None	
MOVLW	k	Move Literal to WREG	1	0000	1110	kkkk	kkkk	None	
MULLW	k	Multiply Literal with WREG	1	0000	1101	kkkk	kkkk	None	
RETLW	k	Return with Literal in WREG	2	0000	1100	kkkk	kkkk	None	
SUBLW	k	Subtract WREG from Literal	1	0000	1000	kkkk	kkkk	C, DC, Z, OV, N	
XORLW	k	Exclusive OR Literal with WREG	1	0000	1010	kkkk	kkkk	Z, N	

REGISTER 5-2: STATUS REGISTER

U-0	U-0	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
—	—	—	N	OV	Z	DC ⁽¹⁾	C ⁽²⁾
bit 7			bit 6				